CHAI SHOP MANAGEMENT SYSTEM

A Comprehensive Salesforce-Based CRM Documentation

Project Documentation

This document presents the analysis, design, planning, configuration, automation, testing, security, reporting, limitations, and future scope of a Chai Shop Management solution based on the supplied project material.

Bachelor of Computer Applications (BCA)

Semester V

The Chai Shop & Co

Document Scope and Basis

This documentation is prepared from the supplied Chai Shop project material. The source describes a Salesforce-based CRM application for The Chai Shop & Co, including customer and account management, Online and Offline customer classification, order and billing-related calculations, automated email communication, reports, dashboards, validation rules, Salesforce Flows, security controls, project planning, testing, advantages, limitations, conclusion, and future scope.

The document keeps those core concepts and expands them into a structured academic project report. Where a section discusses a possible enhancement or a design interpretation rather than a demonstrated feature in the supplied material, it is presented as a proposed or future-oriented item rather than as a verified implementation.

The purpose is to provide a substantial documentation package that can be used as the written project record, subject to the student team adding institution-specific declarations, screenshots, actual configuration IDs, test evidence, and other mandatory submission material if required by the department.

Table of Contents

1. Introduction and Project Overview; 2. Background and Existing Process; 3. Ideation Phase; 4. Empathy Map and User Analysis; 5. Problem Definition; 6. Objectives and Scope; 7. Requirement Analysis; 8. Functional Requirements; 9. Non-Functional Requirements; 10. User Stories and Acceptance Criteria; 11. Customer Journey; 12. Data Flow; 13. System Architecture; 14. Technology Stack; 15. Salesforce Data Model; 16. Objects and Fields; 17. Relationships; 18. User Interface and Navigation; 19. Validation Rules; 20. Formula and Billing Logic; 21. Automation and Flow Design; 22. Email Communication; 23. Security and Access Control; 24. Reports; 25. Dashboards; 26. Project Planning; 27. Product Backlog; 28. Sprint Schedule; 29. Development Milestones; 30. Implementation Approach; 31. Configuration Procedure; 32. Data Management; 33. Testing Strategy; 34. Functional Testing; 35. Performance Testing; 36. Security Testing; 37. Usability Testing; 38. Defect Management; 39. Deployment and Maintenance; 40. Advantages; 41. Limitations; 42. Risk Analysis; 43. Data Quality; 44. Business Benefits; 45. Learning Outcomes; 46. Case Scenarios; 47. Sample Workflows; 48. Sample Reports; 49. Sample Dashboard Design; 50. Future Scope; 51. Mobile and Integration Scope; 52. AI and Analytics Scope; 53. Inventory and Multi-Branch Scope; 54. Conclusion; 55. References and Appendix material.

1. Introduction

The Chai Shop Management project is conceived as a digital solution for organizing customer, order, billing-related, communication, and business information in a centralized environment. The supplied project material identifies Salesforce as the principal platform and describes a CRM application that combines structured data management with validation, automation, reporting, and dashboard capabilities.

Traditional shop operations can depend heavily on handwritten notes, spreadsheets, repeated calculations, and manual communication. Such methods may be workable for a small operation but become harder to control as the number of customers, orders, products, and transactions increases. The proposed CRM approach addresses this operational challenge by keeping relevant records in one platform and by using configured Salesforce capabilities for repetitive activities.

The application is intended to support different user perspectives, including sales representatives, store managers, administrators, business analysts, and customers. Each user group interacts with the business information differently, so the system emphasizes role-appropriate access and organized workflows.

2. Project Overview

The project overview in the supplied material describes a Chai Shop digital platform intended to make ordering and shop management more convenient. Customers can view menu items, select products, specify quantities, review totals, place orders, and follow order status. The shop side can manage products, prices, availability, customer information, and incoming orders.

For the CRM-oriented implementation, the focus extends beyond ordering to customer relationship management. Customer information can be maintained centrally, customers can be classified through defined record types, and business users can use reports and dashboards to understand the information stored in Salesforce.

The project therefore combines operational organization with CRM principles. Instead of treating every order as an isolated transaction, the system provides a structured record of the customer and related business activity, creating a foundation for consistent service and future expansion.

3. Background and Existing Process

Before a centralized CRM is introduced, a chai shop may maintain customer details, product information, orders, and sales records in separate notebooks, spreadsheets, or files. The supplied problem statements emphasize that scattered records make it difficult to obtain timely information and can increase the possibility of manual errors.

A manual process also creates repetitive work. A sales representative may enter customer details, calculate quantities, prepare a bill, update status, and communicate with the customer. A manager may then consolidate the same information for reporting. The same data may therefore be handled multiple times.

The proposed Salesforce environment seeks to reduce this repetition by storing information in objects, applying validation rules, using flows for defined automation, and generating reports from centralized records.

4. Ideation Phase

The ideation phase identifies multiple possible solutions, including manual management, spreadsheet-based management, a standalone web CRM, a cloud CRM, a Salesforce-based CRM, automated sales calculations, automated notifications, and reporting dashboards.

The selected idea in the source is a Salesforce-Based CRM Application for The Chai Shop & Co. The selection aligns with the need for centralized customer and account management, customer classification, automated calculations, business reporting, and automated communication.

The ideation stage is important because it converts a broad business problem into a platform-oriented solution. It also establishes a foundation for later requirement analysis, architecture, implementation planning, and testing.

5. Empathy Map and User Analysis

The source identifies sales representatives, store managers, system administrators, business analysts, and customers as important users. Their needs differ, but all depend on accurate and accessible information.

Users may think about reliable customer records, efficient order processing, accurate calculations, and useful business insights. They may hear feedback about service and sales performance, see records and manual calculations, and perform tasks such as entering customer data, processing orders, generating reports, or communicating with customers.

The major pains identified in the source include manual calculations, data-entry errors, scattered information, delayed reports, repetitive work, and insufficiently timely communication. Expected gains include centralized information, automated calculations, organized records, reports, dashboards, and automated notifications.

6. Problem Definition

For a sales representative, the problem is the difficulty of managing customer information, orders, quantities, and billing efficiently when manual data entry and calculations are involved.

For a store manager, the challenge is monitoring sales performance, customer records, orders, and business activity when information may be distributed across different records. This can make analysis slower and less consistent.

For a customer, the concern is receiving timely updates, personalized offers, and dependable service. The CRM concept addresses these concerns through structured customer records and configured communication mechanisms.

7. Objectives

The project aims to establish a centralized Salesforce CRM environment for The Chai Shop & Co. A major objective is to organize customer, employer or enterprise, account, order, billing-related, and sales information.

Another objective is to reduce manual effort through validation rules, formula-based calculations, and Salesforce Flows. The system also aims to support customer communication through configured email notifications and to provide management visibility through reports and dashboards.

The objectives can be summarized as data organization, transaction accuracy, automation, communication, controlled access, reporting, and a scalable foundation for future enhancements.

• Centralize CRM information

• Reduce repetitive manual processing

• Improve consistency of entered data

• Automate defined calculations and notifications

• Provide structured reports and dashboards

• Support role-based access and controlled visibility

8. Project Scope

The core scope covers CRM data management, customer classification, order-related information, billing-related calculations, validation, automation, email communication, reports, dashboards, and access controls described in the source material.

The scope does not automatically mean that every possible retail feature is implemented. Features such as advanced inventory, payment-gateway integration, WhatsApp messaging, AI prediction, and multi-branch operation are identified in the source as future possibilities rather than confirmed core implementation.

Keeping the scope explicit helps the project remain academically defensible. The implementation should be evaluated against the documented requirements instead of against features that were only proposed for future development.

9. Requirement Analysis

Requirement analysis translates business needs into functions and quality expectations. The supplied documentation separates functional requirements from non-functional requirements and provides examples for data management, validation, automation, billing, reporting, security, performance, availability, and scalability.

Functional requirements describe what the application should do. Non-functional requirements describe how well the application should behave with respect to security, reliability, performance, availability, usability, and growth.

This separation helps the project team create test cases that verify both individual features and broader system qualities.

10. Functional Requirements

The core functional requirements include creating and maintaining customer information, supporting employer or enterprise records, managing account information, supporting customer record types, validating data, calculating configured order or billing values, automating selected processes, sending configured notifications, generating reports, and presenting dashboards.

Each requirement should have a clear trigger, input, processing rule, and expected outcome. This makes the requirement testable and reduces ambiguity during configuration and evaluation.

The requirements are implemented through Salesforce objects, fields, record types, validation rules, formula fields, flows, email templates, reports, and dashboards as described by the source.

11. Non-Functional Requirements

The source defines security, reliability, performance, availability, and scalability requirements. Security covers role-based access, authentication, object-level security, field-level security, record-level access, profiles, and permission sets.

Reliability concerns dependable storage, validation, automation, error handling, and recovery. Performance concerns responsive pages, efficient reports, dashboards, and automation. Availability concerns access for authorized users during business operations. Scalability concerns the ability to accommodate more customers, users, records, reports, and potentially branches.

These requirements are important because a CRM is not useful merely because it can store data; it must also provide controlled and dependable access to that data.

12. User Stories

The project source provides user-story-oriented planning for customer management, application structure, validation, automation, billing, reporting, and dashboards. User stories express a capability from the perspective of a user or system and connect directly to acceptance criteria.

Examples include a user wanting to manage customer information, a system validating customer and order data, a system automating notifications, and a manager requesting reports and dashboards.

Story points and sprint assignments in the supplied project plan provide a lightweight Agile-inspired planning model.

13. Acceptance Criteria

Acceptance criteria define the observable result required for a story to be considered complete. For customer creation, valid data should result in a saved record. For validation, invalid input should be rejected according to configured rules. For automation, a flow should execute when its defined conditions are satisfied.

For reporting, required CRM information should appear in the appropriate report. For dashboards, dashboard components should correctly represent the underlying report data.

Acceptance criteria should be checked during testing and documented with evidence such as test results or screenshots where required by the academic submission.

14. Customer Journey

The customer journey described by the project begins with discovering the menu and available products. The customer can register or log in, browse items, search for a product, choose quantities, customize available options, review the cart, check the total, and place an order.

After order placement, the customer can view an order status such as received, preparing, ready, or completed. These status stages provide a simple way to communicate progress and reduce uncertainty.

From the shop perspective, the same journey generates operational records that can be used to monitor orders, products, customer details, and business activity.

15. Data Flow Diagram

The source describes a high-level flow: User to Salesforce Lightning UI, then CRM Objects, Validation Rules, Salesforce Flows, Updated Records, Reports, Dashboards, and finally Business Decision.

The user supplies or requests information through the interface. CRM objects store the relevant records. Validation rules check configured requirements. Flows perform defined automated actions. Reports organize stored information, while dashboards present selected metrics visually.

The flow demonstrates how operational data can become management information. It also shows why validation and automation should occur before reporting: the quality of analytics depends on the quality and consistency of stored records.

16. System Architecture

The architecture is divided into a presentation layer, application and automation layer, data layer, security layer, and analytics layer. Salesforce Lightning provides the user interface. Salesforce Flows and business logic handle automation and validation. Salesforce objects provide storage. Profiles, permission sets, and sharing controls manage access. Reports and dashboards provide analysis.

This separation supports modularity and maintainability. Changes to a report do not necessarily require changes to the data model, while changes to a validation rule can be made without redesigning the entire interface.

The architecture is cloud-oriented and uses Salesforce platform capabilities rather than a separately deployed traditional three-tier application.

17. Technology Stack

The supplied technology stack centers on Salesforce Lightning Experience, Salesforce Objects, Salesforce Flow, Validation Rules, Formula Fields, Email Templates, Reports, and Dashboards. Salesforce Developer or Trailhead Playground environments are identified as development and configuration environments.

The platform approach reduces the need to build every infrastructure component from scratch. Instead, the project focuses on configuring objects, fields, relationships, automation, security, and analytics according to the business requirements.

The technology choices are closely aligned with the selected solution and the educational goal of learning Salesforce CRM configuration concepts.

18. Salesforce Data Model

The data model organizes information into Salesforce standard and custom objects. The supplied project refers to customer, employer or enterprise, account, product, order, billing-related, and sales information. Exact object names and field definitions should follow the team’s actual Salesforce configuration.

A good data model avoids unnecessary duplication and keeps related records connected through appropriate relationships. Customer information should be maintained consistently, while order information should reference the relevant customer and product data where applicable.

The data model is the foundation for validation, automation, reporting, and dashboards; therefore, changes to objects or relationships can affect multiple project components.

19. Objects and Fields

Fields represent individual attributes within records. The project requires fields for customer information, classification, contact details, order information, quantities, billing-related values, and other business data.

Picklists and global value sets can standardize values such as customer type or order status. Formula fields can derive configured values without requiring manual entry. Required fields ensure essential information is present before a record is saved.

Field design should balance completeness with usability. Collecting unnecessary information increases entry effort, while collecting too little information limits reporting and customer service.

20. Relationships

Relationships connect business entities so users can navigate from one record to related information. A customer can be associated with orders, and orders can contain relevant product or billing information according to the configured data model.

Relationship design should support the project’s reporting needs. If management needs customer-level sales analysis, the model must preserve a dependable connection between customer records and sales or order records.

The exact relationship type and field configuration should be verified against the actual Salesforce org before final submission.

21. User Interface and Navigation

Salesforce Lightning provides the interface for accessing records, forms, reports, dashboards, and other CRM features. The source includes custom tabs and a Lightning application as project activities.

A practical navigation design places frequently used records and analytics where the relevant users can reach them easily. Sales users may need customer and order views, while managers may need reports and dashboards.

Page layouts and record types can tailor the data-entry experience. The interface should avoid exposing fields that a particular role does not need.

22. Validation Rules

Validation rules help prevent incorrect or incomplete information. The supplied testing section specifically mentions phone, PAN, and Aadhaar validation, along with required fields and record-type behavior.

A validation rule should clearly express a business condition under which a record must not be saved. Error messages should be understandable enough for the user to correct the input.

Validation is a preventive control. It is generally better to stop invalid information at entry than to allow it into reports and attempt to clean it later.

23. Formula and Billing Logic

The source identifies formula-based billing calculations as part of the CRM configuration. Formula fields can calculate configured values from stored inputs such as quantity and price.

A typical conceptual calculation is total amount equals unit price multiplied by quantity, subject to the actual formula configured in the project. The documentation should describe the real formula used rather than claiming a formula that was not implemented.

Automated calculation reduces repetitive arithmetic and creates consistent outcomes for records processed under the same rule.

24. Automation and Flow Design

Salesforce Flows are used in the source to automate record updates, processing, and notifications. Automation can be triggered by defined record conditions and can update related information or initiate communication.

A flow should have a clear entry condition, decision logic where necessary, actions, and a predictable end state. Testing should include both the condition where the flow should run and conditions where it should not run.

Good automation design also considers error handling and avoids unnecessary processing. Each flow should have a documented purpose so that future administrators can maintain it.

25. Email Communication

The project includes email templates and automated birthday email notifications with personalized special offers. Email automation is intended to improve customer communication and engagement.

Templates provide consistent message structure while flows or other configured automation determine when a message should be sent. Before production use, the organization should verify recipient data, opt-in requirements, sender settings, and delivery behavior.

Communication features should be tested using controlled sample records to ensure that messages are triggered only under the intended conditions.

26. Security and Access Control

Security is addressed through profiles, permission sets, object-level permissions, field-level security, and record-level access. The goal is to ensure that users can access only the information needed for their responsibilities.

Sensitive billing-related information should receive appropriate protection. A sales representative may need customer and order information without necessarily requiring unrestricted access to every administrative field.

Security testing should verify both positive access and negative access: the right user should be able to perform required work, while an unauthorized user should be prevented from accessing restricted information.

27. Reports

Reports convert stored CRM information into structured business views. The source identifies customer, order, sales, and business analysis reports as important outputs.

A report definition should specify its purpose, data source, filters, grouping, and useful columns. For example, a sales report may group transactions by period or customer if those fields exist in the configured data model.

Reports should be validated against known sample records so that totals and filters can be checked.

28. Dashboards

Dashboards provide visual summaries of selected report data. The source positions dashboards as a management aid for understanding important CRM and business metrics.

A dashboard should contain only meaningful components. Possible components include sales summaries, order status distribution, customer classification, and other metrics supported by the actual data.

Dashboard testing should verify that each component reflects its source report correctly and that filters behave as intended.

29. Project Planning

The supplied project follows an Agile-inspired approach in which work is divided into manageable sprints and milestones. Major planning activities include environment setup, object creation, fields and relationships, tabs, Lightning application configuration, validation, automation, email templates, reporting, dashboards, testing, and final validation.

Incremental development allows each group of features to be configured and reviewed before the next group is added. This reduces the risk of discovering fundamental data-model issues only at the end of the project.

The planning model also creates a basis for documenting responsibilities and tracking completion.

30. Product Backlog

The project backlog includes stories for development environment setup, CRM structure, data validation, automation, billing automation, reporting, and dashboards. Story points are used to estimate relative effort.

The supplied plan totals 46 story points across six sprints of six days each. Sprint allocation should be treated as the project plan described in the source, while actual completion should be documented from the team’s real work.

A backlog can be updated during implementation if requirements change, but any changes should be recorded so the final report remains consistent with the delivered system.

31. Sprint Schedule

The source proposes six sprints. Sprint 1 covers environment, objects, and the basic data model. Sprint 2 covers tabs, Lightning application structure, fields, relationships, layouts, and record types. Sprint 3 covers validation, email templates, and Flow automation.

Sprint 4 focuses on billing calculations and related automation. Sprint 5 covers reports and data analysis. Sprint 6 covers dashboards, testing, and final refinement. Each sprint is planned for six days.

This schedule creates a logical progression from data foundation to automation and then analytics.

32. Development Milestones

The supplied milestone list contains development environment setup, object creation, custom tabs, Lightning application creation, global picklists, fields and relationships, page layouts, record types, validation rules, email templates, Salesforce Flows, reports and dashboards, and final testing.

Each milestone should produce a tangible configuration result. For example, the object milestone should result in the required objects, while the validation milestone should result in documented rules and test cases.

Milestone-based documentation is useful during project reviews because it shows how the application moved from concept to configured CRM solution.

33. Implementation Approach

Implementation is primarily configuration-oriented. The project uses Salesforce platform features to create the data model, interface, validation, automation, communication, reporting, and security controls.

The implementation should proceed from the data model outward. Objects and fields should be stable before automation is built, because flows and formulas depend on field names and relationships. Reports should be created after the underlying records and fields are available.

Final testing should be performed after integration of the individual features, because a feature can work alone but still fail when combined with other automation or security controls.

34. Configuration Procedure

A repeatable configuration procedure begins by preparing the Salesforce development environment. The next steps are object and field creation, relationships, picklists, tabs, Lightning app navigation, layouts, record types, validation rules, formula fields, email templates, flows, reports, dashboards, security, and testing.

Configuration should be documented with the exact names used in the Salesforce org. This prevents confusion when another team member or evaluator attempts to verify the implementation.

Where screenshots are required, the team can insert screenshots after each major configuration section without changing the conceptual structure of this report.

35. Data Management

Data management includes creating, updating, validating, and maintaining customer, account, order, product, billing-related, and sales records. Standardized field values improve consistency and make reporting more reliable.

Sample or project-specific CRM data is identified in the source for testing and demonstration. The appendix lists customer records, employer or enterprise records, product and order information, billing-related information, and sales records.

Data maintenance should include attention to duplicates, invalid values, incomplete records, and obsolete information.

36. Testing Strategy

Testing verifies whether the configured application satisfies its requirements. The source includes functional testing, performance testing, dashboard verification, and user-access testing.

A complete strategy should cover individual features and end-to-end workflows. Functional tests verify expected behavior. Negative tests verify rejection of invalid inputs. Security tests verify permissions. Performance observations verify acceptable responsiveness.

Test evidence should record test ID, feature, input, expected result, actual result, status, and remarks.

37. Functional Testing

The supplied functional test summary includes customer record creation, employer or enterprise records, phone validation, PAN validation, Aadhaar validation, record types, page layouts, billing formula, flow automation, email notification, reports, and dashboards. All are described as having passed in the supplied source.

For an academic final report, the team should preserve only test results that correspond to tests actually executed. If a test was not performed in the final org, it should be marked as pending or planned rather than represented as completed.

Functional testing is strongest when both valid and invalid scenarios are covered.

38. Performance Testing

Performance testing considers page loading, record creation and editing, report generation, dashboard display, Flow execution, and usability as record volume increases.

The supplied source intentionally states that performance observations can be documented based on actual testing performed. Therefore, exact response times should not be invented. Instead, the team can add measured observations from its Salesforce environment if available.

Performance evaluation should focus on whether the configured solution remains practical for the expected workload.

39. Security Testing

Security testing verifies that profiles, permission sets, field permissions, object permissions, and record access behave according to the intended roles.

Positive tests confirm that authorized users can perform required tasks. Negative tests confirm that restricted users cannot view or modify protected information.

Security should be tested after the data model and user roles are configured, because access rules depend on the objects, fields, and records they protect.

40. Usability Testing

Usability concerns how easily users can understand and complete common tasks. Lightning navigation, page layouts, required fields, meaningful labels, and clear error messages all contribute to usability.

Representative tasks can include creating a customer, viewing an order, checking a report, and opening a dashboard. Users should be able to complete these tasks without unnecessary navigation or confusing terminology.

Usability feedback can be converted into configuration improvements such as rearranging fields, refining labels, or simplifying navigation.

41. Defect Management

Defect management provides a controlled way to record problems found during testing. Each defect should include a description, affected feature, steps to reproduce, expected behavior, actual behavior, priority, status, and resolution.

Typical CRM defects may involve incorrect validation conditions, a flow not firing, a report filter excluding records, a dashboard component showing unexpected values, or a permission preventing a legitimate task.

After correction, the affected test should be repeated to confirm the defect is resolved and no related behavior has been broken.

42. Deployment and Maintenance

After configuration and testing, the project can be prepared for the target Salesforce environment according to the team’s deployment method. The supplied source does not prescribe a single deployment mechanism, so the final report should document the actual method used.

Maintenance includes updating fields, rules, flows, reports, dashboards, and user permissions when business requirements change. A controlled change process helps prevent accidental disruption to existing automation.

Documentation should be kept current so that future maintainers understand the purpose of each major configuration item.

43. Advantages

The source identifies centralized data management, reduced manual work, improved data accuracy, automated calculations, improved customer communication, better business visibility, controlled data access, and cloud-based accessibility as advantages.

These benefits are closely connected to the platform capabilities used in the project. Validation improves input consistency, formulas reduce manual arithmetic, flows reduce repetitive processing, and reports and dashboards organize business information.

The actual degree of benefit depends on the quality of configuration, data, user adoption, and operational processes.

44. Limitations

The supplied documentation identifies Salesforce cost, internet dependency, limited offline functionality, platform dependency, configuration complexity, and data migration effort as limitations.

These limitations should be considered when evaluating the project. A cloud CRM can improve centralized access but still depends on network availability. Salesforce reduces infrastructure work but creates dependency on the platform and its licensing model.

Data migration can also be substantial when an organization has legacy spreadsheets or inconsistent records. Data cleaning and mapping may be required before importing information.

45. Risk Analysis

Project risks include incorrect data design, incomplete requirements, configuration errors, automation failures, security misconfiguration, inconsistent sample data, and insufficient testing.

A practical mitigation approach is to validate the data model early, maintain a requirement-to-test mapping, test automation with controlled records, review permissions by role, and document configuration changes.

Risk analysis should remain proportional to the project. The goal is to identify realistic failure points and prepare practical controls rather than creating an unnecessarily complex risk framework.

46. Data Quality

Data quality is essential because reports and dashboards are only as reliable as the records behind them. Validation rules and standardized picklist values help prevent some classes of errors at entry time.

Additional data-quality practices include avoiding duplicate customer records, using consistent phone formats, validating required identifiers, reviewing outdated records, and monitoring unusual values.

A data-quality review should be performed before final screenshots and reports are captured so that demonstration outputs represent clean sample data.

47. Business Benefits

The CRM approach can support more organized customer service, consistent transaction processing, easier monitoring of orders, and structured business analysis. Managers can use reports and dashboards to view information without manually consolidating every record.

Customer communication can be more systematic when templates and automated notifications are configured. The project also creates a centralized record of customer and business information that can support future enhancements.

The benefits described here are operational objectives of the system; measurable business impact would require real usage data after deployment.

48. Learning Outcomes

The source identifies learning outcomes such as Salesforce CRM application development, standard and custom objects, fields and data types, relationships, picklists, custom tabs, Lightning applications, page layouts, record types, validation rules, email templates, Flow automation, reports, and dashboards.

Students also gain experience in requirement analysis, project planning, testing, documentation, and translating a business problem into a configurable CRM solution.

These skills are transferable to other Salesforce configuration projects because they represent common platform concepts rather than a single shop-specific feature.

49. Case Scenario: New Customer

A new customer record begins with the entry of required customer information. The system checks configured validation rules before saving. The customer classification can be selected using the supported record-type or field configuration.

Once the record is stored, it becomes available for related business activities according to the configured relationships and access controls.

This scenario demonstrates the interaction among user interface, data fields, validation, security, and record storage.

50. Case Scenario: Order Processing

An order scenario begins when an order is recorded against the relevant customer and selected product information. Quantity and price-related values are captured according to the data model. A configured formula can calculate the billing-related amount.

Automation may update records or trigger communication when defined conditions are satisfied. The resulting record can then appear in reports and dashboard components.

The exact sequence should follow the team’s actual Salesforce configuration; this section documents the conceptual process represented by the source.

51. Case Scenario: Customer Communication

A customer communication scenario can use a configured email template and an automation condition such as a birthday-related date or another supported business event.

The communication flow should verify the intended recipient and condition before sending. Test records should be used during development to avoid unintended messages.

This capability illustrates how CRM data can support relationship-oriented communication instead of serving only as a storage system.

52. Case Scenario: Manager Reporting

A manager can use reports to inspect customer, order, and sales information. The manager can then access dashboard components that summarize selected metrics.

The value of this workflow depends on clean source records, suitable report filters, and meaningful dashboard design. Reports should therefore be validated before being added to a dashboard.

This scenario connects the operational data layer to management-level analysis.

53. Sample Workflow Documentation

A workflow description should state the trigger, conditions, actions, and expected outcome. For example, a record-triggered Flow can be documented as: trigger on the relevant record event, check the configured condition, update or create the required information, and optionally initiate a notification.

The actual Flow name, entry condition, and actions should be inserted from the project org. The documentation should not claim an action that the implementation does not contain.

Clear workflow documentation makes troubleshooting easier because administrators can identify which automation is responsible for a change.

54. Sample Report Design

A report specification can include report purpose, source object, columns, filters, grouping, and intended audience. A customer report may focus on classification and contact information, while a sales report may focus on order and billing-related values.

Reports should be kept focused. A report containing every available field can become difficult to interpret.

Before final submission, each report should be checked using known sample records so that filters and totals are correct.

55. Sample Dashboard Design

A dashboard can combine a small number of useful components derived from validated reports. Possible categories include sales summary, order status, customer classification, and other metrics supported by the project data.

Dashboard design should prioritize readability. Labels should be understandable, charts should match the type of information being displayed, and unnecessary components should be avoided.

The dashboard is a presentation layer for business analysis rather than a separate data store. Any discrepancy should therefore be traced back to the underlying report or records.

56. Future Scope: Mobile CRM

The source proposes mobile CRM optimization so sales representatives and managers can access important information while working remotely. A mobile-focused enhancement could prioritize customer lookup, order review, status updates, and essential reports.

The implementation would need usability testing on supported mobile experiences and careful review of which fields and actions are practical on a smaller screen.

This is a future enhancement and should not be represented as a completed feature unless it has actually been configured and tested.

57. Future Scope: AI and Analytics

The source identifies AI-based sales prediction as a possible future enhancement. Such a feature could explore historical sales patterns, demand trends, and data-driven insights using appropriate Salesforce-supported capabilities.

Any predictive system would require sufficiently consistent historical data and a clearly defined prediction target. The quality of the output would depend on the data used and the design of the analytical method.

For the current project, AI is therefore best treated as future scope rather than as an existing implementation unless the team has separately configured and evaluated it.

58. Future Scope: Notifications and Integration

The source proposes WhatsApp or SMS notifications in addition to email. Such integration would require an appropriate external communication service and a supported integration mechanism.

Other future integrations could connect Salesforce with accounting, payment, inventory, communication, or related business systems through suitable APIs and technologies.

Integration should be introduced only after security, data mapping, error handling, and privacy requirements are defined.

59. Future Scope: Inventory and Multi-Branch

Inventory management could track product quantities, stock availability, stock movement, and reorder requirements. Multi-branch support could extend the data model and access controls to handle several business locations.

These enhancements would increase the complexity of the data model and reporting requirements. Branch-specific visibility, shared product information, and consolidated management reporting would need to be designed carefully.

The source identifies these capabilities as future scope, so they should be described as proposed extensions rather than current functions.

60. Advanced Customer Profiling

The future scope includes advanced customer profiling using purchase history, preferences, interaction history, and segmentation. Such information could support more personalized customer engagement if collected and managed appropriately.

Customer profiling should follow applicable privacy and data-governance requirements. Only information needed for a legitimate business purpose should be collected and retained.

A mature CRM can use customer history to understand service patterns, but the quality of those insights depends on accurate and consistently maintained records.

61. Conclusion

The Chai Shop & Co CRM Application demonstrates how Salesforce can be used to organize customer and business information in a centralized cloud environment. The supplied project combines data management, validation, automation, billing-related calculations, email notifications, reports, dashboards, and security controls.

The project provides a structured example of applying CRM concepts to a small-business context. It also gives practical exposure to Salesforce objects, fields, relationships, record types, validation rules, formulas, flows, email templates, profiles, permission sets, reports, and dashboards.

Overall, the documented solution provides a foundation for organized CRM operations while leaving clear opportunities for future extensions such as mobile access, AI-supported analytics, additional notification channels, inventory, integrations, and multi-branch management.

62. Final Documentation Checklist

Before submitting the project, the team should verify that the report matches the actual Salesforce configuration. Names of objects, fields, flows, reports, dashboards, profiles, and permission sets should be checked against the org.

Screenshots should be added where the college requires implementation evidence. Test results should represent tests that were actually performed. Any future-scope feature should remain clearly separated from implemented functionality.

The final package should also contain the institution’s required certificate, declaration, acknowledgement, team details, references, and other front-matter sections if they are part of the official submission format.

63. Requirement Traceability Matrix

The following matrix provides a documentation-level mapping between major requirements and Salesforce capabilities. It should be aligned with the actual configuration before submission.

Requirement Area	Primary Capability	Evidence
Customer Management	Objects, fields, layouts	Customer records
Data Validation	Validation rules	Valid/invalid test cases
Billing Calculation	Formula field / configured logic	Billing test case
Automation	Salesforce Flow	Flow execution test
Communication	Email template + automation	Notification test
Reporting	Salesforce Reports	Report output
Analytics	Dashboards	Dashboard verification
Security	Profiles, permission sets, sharing	Access tests

64. Test Case Template

Use the following structure for each final test case. The team can duplicate rows or add screenshots as evidence.

Test ID	Feature	Input / Condition	Expected Result	Status
TC-01	Customer creation	Valid required data	Record saved	Actual result
TC-02	Phone validation	Invalid phone value	Validation error	Actual result
TC-03	Billing	Configured quantity and price	Correct configured amount	Actual result
TC-04	Flow	Condition satisfied	Flow actions execute	Actual result
TC-05	Report	Known sample records	Expected records displayed	Actual result
TC-06	Dashboard	Validated source report	Component reflects report	Actual result
TC-07	Security	Restricted user	Access denied where configured	Actual result

65. Appendix: Tools and Technologies

The source identifies the following tools and technologies for the project. The exact editions or account configurations should be recorded from the environment used by the team.

Tool / Technology	Purpose
Salesforce Developer / Trailhead Playground	Development and configuration environment
Salesforce Lightning Experience	CRM user interface
Salesforce Objects	Data storage and management
Salesforce Flow	Business process automation
Validation Rules	Data validation
Formula Fields	Automated calculations
Email Templates	Customer communication
Reports	Data analysis
Dashboards	Data visualization

66. Appendix: Sample Data Categories

The supplied project identifies sample or project-specific CRM data for testing and demonstration. This page organizes those categories for documentation purposes.

Category	Examples of Information
Customer Records	Customer identity and contact information
Employer / Enterprise Records	Business or organizational information
Product Information	Product name, category, price, availability
Order Information	Customer reference, product reference, quantity, status
Billing-Related Information	Configured price and calculated amount
Sales Records	Transaction information used for reporting

67. Appendix: Glossary

CRM means Customer Relationship Management. Salesforce Objects are data structures used to store records. A Flow is a Salesforce automation mechanism. A Validation Rule checks whether record data satisfies a defined condition. A Formula Field calculates a value from configured expressions. A Record Type can support different business processes or categories. A Report organizes Salesforce data for analysis. A Dashboard presents selected report information visually. A Profile and Permission Set contribute to user access control.

These terms are used throughout the project documentation and should be kept consistent with the terminology used in the Salesforce environment.

68. Appendix: Suggested Evidence Pages

To make the final academic submission stronger, the team can insert actual screenshots in this section. Suggested evidence includes the Salesforce home or Lightning application, object configuration, fields, relationships, record types, validation rules, formula field, Flow canvas, email template, profiles or permission sets, reports, dashboards, and test records.

Only screenshots from the actual project environment should be inserted. Each screenshot should have a short caption explaining what it proves. Avoid including passwords, security tokens, personal contact information, or other sensitive data in screenshots.

69. Appendix: Project Review Questions

Review questions can be used during a project demonstration: Why was Salesforce selected? What are the main users? What problem does centralized CRM solve? Which objects store customer and order information? How are invalid values prevented? How is billing-related calculation automated? Which processes use Flow? How are emails triggered? How is access controlled? Which reports and dashboards are provided? What are the main limitations? Which features belong to future scope?

Preparing concise answers to these questions helps the team demonstrate understanding rather than simply presenting configuration screens.

70. Closing Note

This 60+ page documentation package is designed as an expanded academic report for the Chai Shop Management / The Chai Shop & Co Salesforce CRM project. It preserves the supplied project's main structure and terminology while expanding the explanations needed for a longer documentation submission.

For final college submission, the most important final step is to synchronize the report with the actual Salesforce implementation. Replace generic placeholders with the team's real screenshots, configuration names, test evidence, and institution-required front matter.

The resulting report can then function as a combined project documentation, implementation explanation, testing record, and future-development reference.